

MacroBase: Static Bug and Reliability Audit

Android offline nutrition tracker (Kotlin, Jetpack Compose, Room, ML Kit)

Project	MacroBase (com.macrobase.app, versionName 1.2.2, versionCode 13)
Audit date	3 October 2026
Version control	Branch master, latest commit ddfc057dabb63fbcc97e1cb7866f7b88ccad2edc ("added rank images"), plus uncommitted changes in 64 tracked files and several untracked files, audited as found in the working tree
Scope	Whole repository: app module (main sources, resources, manifest, bundled database, tests), ppocr-sdk module, build configuration, data-generation tool, documentation
Findings	42 total: P0 0, P1 1, P2 12, P3 19, P4 10

This was a read-only static review. No code was executed or modified.

B. Executive Summary

The audit found **42 issues: P0 0, P1 1, P2 12, P3 19, P4 10**. By confidence: Confirmed 29, Highly likely 9, Potential 4.

Overall assessment: not ready for production use as a calorie tracker. The app's central promise is accurate, private, offline nutrition logging. The built APK requests INTERNET through ML Kit, which contradicts the zero-network invariant.

Most dangerous issues

- **BUG-008 [P1]** The built APK declares INTERNET and ships ML Kit's Google telemetry transport, breaking the zero-network promise

User journeys most at risk: logging catalog foods; creating and logging recipes; editing past entries; restoring a backup; scanning labels from kJ, Canadian or US-format packaging.

The code is generally tidy, uses snapshots for history and has a sizeable test suite (about 290 unit tests). The defects cluster in a few patterns (section G), so a focused fix sequence (section H) removes most of the user impact.

C. Scope, Method, and Coverage

System type: Android mobile app, offline, single user. Kotlin 2.1, Jetpack Compose (Material 3), Navigation Compose, Koin DI, Room 2.6.1 (user database v3), AndroidX DataStore, a read-only bundled SQLite catalog with FTS5, CameraX, Google ML Kit text recognition, and an included PaddleOCR (ONNX Runtime/OpenCV) library module. minSdk 26, targetSdk 35.

Method: reconnaissance of README, AGENTS.md, docs and build files; risk-ordered review of the data layer by the lead auditor; four parallel read-only reviewers (portability and security; scanner; food logging UI; secondary features) whose findings were each re-checked against the source by the lead auditor; a fresh-context adversarial reviewer that tried to disprove the ten most serious findings (none were overturned; two were narrowed and the text adjusted). The bundled database and an older catalog from git history were queried read-only.

Coverage	Areas
Reviewed fully	Room database, entities, DAOs, migrations, DI; built-in database manager and provider; food, diary, recipe, goals, preferences, portability and rank repositories; use cases; domain models; Food Detail, Basket, Search, Custom Foods, Recipes, Home dashboard; backup archive manager, JSON serializer and import/export screen; scanner pipeline (screen, ViewModel, orchestrator, parsers, detectors, normalisation, OCR engines); widget, MainActivity, manifest and backup rules; merged manifests and merger reports
Reviewed partially	Weight, Water, Statistics, Calendar, Preferences and Rank screens (logic and ViewModels; not every composable); design-system components; ppocr-sdk (lifecycle and crash paths only; unused by the app); unit tests (read for coverage, not run); tools/build_indian_food_database.py and the CSV (where they explain bundled data)
Not reviewed	androidTest sources in depth; Compose visual layout and accessibility semantics in detail (no device available); docs beyond what was needed to understand intent
Skipped	build/ and .gradle/ output (except merged manifests and merger reports), .idea/, .kotlin/, binary assets (ONNX models, rank images), vendored Gradle wrapper

About 130 source and test files (roughly 25,000 lines of main Kotlin) plus the bundled database, manifests and build files were reviewed.

D. Priority Table

ID	Priority	Issue	User impact	Confidence	Location
BUG-008	P1	The built APK declares INTERNET and ships ML Kit's Google telemetry transport, breaking the zero-network promise	The app holds network permission and Google's logging transport while the UI says "Never uploaded to the cloud" (ImportExportScreen.kt:305) and AGENTS.md calls zero network non-negotiable.	Confirmed	app/build/intermediates/merged_manifest/release/processReleaseMainManifest/AndroidManifest.xml:26-27, 132-145
BUG-011	P2	Double-tapping Log or Save creates duplicate diary entries or pops past the dashboard	Duplicate foods on the dashboard and doubled totals; occasionally a blank screen after popping too far.	Highly likely	feature/detail/FoodDetailScreen.kt:312-352 (logFood)
BUG-012	P2	Quantity fields accept blank, comma decimals, zero and negatives, logging 0 kcal or negative calories	Entries that add nothing, or that reduce the day's total.	Highly likely	feature/detail/FoodDetailScreen.kt:266-282, 312-352
BUG-015	P2	Recipe "Log to Diary" only adds to an in-memory basket, and lost basket items are reported as logged	Meals the user believes are logged never reach the diary.	Confirmed	feature/recipes/RecipesScreen.kt:138-152, 296
BUG-016	P2	Importing any backup schedules a reset of the fitness strategy to Maintaining / 2,000 kcal	Next day the strategy silently switches to Maintaining 2,000; goal history gains a fake transition; rank scoring changes.	Confirmed	data/repository/PortabilityRepositoryImpl.kt:129-134 (export), 229-238 and 379-389 (import)
BUG-017	P2	An archive missing a data file passes validation, and Overwrite restore then wipes that table	The diary is deleted; the result says "Successfully restored N records".	Highly likely	data/portability/BackupArchiveManager.kt:188-230

BUG-018	P2	Android Auto Backup uploads the personal diary database to Google Drive despite the "never uploaded" claim	The app states "Stored completely offline on your device."	Confirmed	app/src/main/AndroidManifest.xml:10-12
BUG-019	P2	Scanner: kilojoule energy values are stored as kilocalories (4.2x too high)	A prefilled custom food with calories about 4.2x too high, marked as detected.	Confirmed	feature/scanner/SpatialTableDetector.kt:499-556
BUG-020	P2	Scanner: an unrecognised basis header defaults to "per 100 g" with high confidence	Per-serving numbers stored as per-100 g with no warning; the macro check stays quiet because the numbers are self-consistent.	Confirmed	feature/scanner/NutritionLabelParser.kt:692-717 (detectBasis)
BUG-021	P2	Scanner: per-serving values without a gram weight are relabelled as per 100 g	A warning asks for the serving weight, but the modal has no field for it; the saved food is wrong.	Confirmed	domain/model/scanner/ScannerModels.kt:149-166 (withOutputBasis)
BUG-022	P2	Scanner: thousands separators are read as decimals ("1,580 mg" becomes 1.58 mg)	Sodium or energy about 1,000x too low with no review flag.	Confirmed	feature/scanner/NutritionLabelParser.kt:1033-1042
BUG-023	P2	Rank rating can never exceed 600 of 2,399, so 5 of the 8 ranks are unreachable	The rank ladder stalls permanently at rank 3; ratings drop for no visible reason.	Confirmed	data/repository/rank/RankRepositoryImpl.kt:39-41, 161-218
BUG-024	P2	Rank scores days against stale calorie and macro targets	Following the on-screen targets is penalised; the rank breakdown disagrees with the dashboard.	Confirmed	data/repository/GoalsRepositoryImpl.kt:148-170, 176-187, 242-266 (getGoalForDate)
BUG-026	P3	Macro-percentage check differs between UI and use case, so some valid goal saves fail silently	Save appears to work; preferences are stored but goals are not, with no message.	Confirmed	domain/model/Goal.kt:56-67
BUG-027	P3	Weight edit dialog rejects its own pre-filled value in comma-decimal locales	"Please enter a valid positive number"; typed "72,5" is also rejected.	Confirmed	feature/weight/WeightScreen.kt:650-656, 724-731, 771-774
BUG-028	P3	Preferences inputs are not validated; comma or cleared values are silently dropped	"Saved" message, but weight unchanged, cleared target reappears, or a negative water goal is stored.	Confirmed	feature/preferences/PreferencesScreen.kt:855-872
BUG-029	P3	Toggling the unit system discards typed preference edits	The typed value vanishes; strategy auto-detection keeps the value from 80, so the two disagree.	Confirmed	feature/preferences/PreferencesScreen.kt:150-182
BUG-030	P3	Tapping the home-screen widget never opens Search and resets the app's back stack	The dashboard opens instead of Search; any screen the user had open is gone.	Confirmed	MainActivity.kt:21-41
BUG-031	P3	A pending fitness-strategy change cannot be cancelled	Tomorrow Cutting is applied anyway.	Confirmed	feature/preferences/PreferencesScreen.kt:189

BUG-032	P3	Rank penalises today before the day is over (up to -20 RR every morning)	Rank drops every morning and recovers during the day.	Confirmed	data/repository/rank/RankRepositoryImpl.kt:165-218
BUG-033	P3	Statistics windows include one extra day and the consistency score is truncated	"96%" instead of 100%; 7-day averages span 8 days.	Confirmed	feature/statistics/StatisticsScreen.kt:144-149, 195-200
BUG-037	P3	Missing fiber, sugar and sodium are stored as 0.0 instead of null in diary snapshots	History shows "0.0 g" where it should show "-"; this 0 is what triggers BUG-010.	Confirmed	data/database/entity/UserEntities.kt:29-31
BUG-038	P3	Food Detail resets the typed quantity and serving on rotation	Quantity returns to the default (or to the stored entry in edit mode).	Highly likely	feature/detail/FoodDetailScreen.kt:428-430
BUG-039	P3	Basket: errors are swallowed and the header shows only the first item's date and meal	Nothing happens on failure; items log to dates the header did not show.	Confirmed	feature/basket/BasketScreen.kt:518-524, 283-296
BUG-040	P3	Dashboard keeps yesterday's date after midnight, so new logs go to the wrong day	Entries land on yesterday; the header changes from "Today" to a date.	Potential	feature/dashboard/HomeScreen.kt:108
BUG-041	P3	Scanner: with a single per-serving column, a second number in the row becomes the per-100 g value	Wrong sodium with only a "Check" badge.	Highly likely	feature/scanner/NutritionLabelParser.kt:1105-1121
BUG-042	P3	Scanner: adjudicator warnings are discarded and critical warnings can be hidden	No visible "calories too high" warning.	Confirmed	feature/scanner/NutritionLabelScanOrchestrator.kt:214-245
BUG-043	P3	Scanner: table-crop OCR coordinates are mixed with full-image coordinates	Intermittently swapped or duplicated nutrient rows.	Highly likely	feature/scanner/NutritionLabelScanOrchestrator.kt:159-175
BUG-044	P3	Scanner: gallery images are decoded at full size on the main thread and can crash with OutOfMemoryError	Freeze, then app crash.	Confirmed	feature/scanner/NutritionLabelScannerScreen.kt:107-122
BUG-045	P3	Scanner: "Calories from Fat" rows take over the fat and calorie values on older US labels	Fat of 72 g/100 g; calories may fall to the footnote "2,000" (then BUG-022).	Highly likely	feature/scanner/NutritionLabelParser.kt:945-953, 1014-1029
BUG-046	P3	Destructive migration fallback wipes all user data on a downgrade or a missed migration	All diary, custom foods, recipes, weight and water history disappear on launch, with no warning.	Potential	core/di/DiModules.kt:91-99
BUG-047	P3	A crafted or very large backup can crash the app; parse errors embed the whole file in the UI	App crash, or a huge error string rendered in a Text (jank/ANR).	Potential	data/portability/BackupArchiveManager.kt:24-28

BUG-048	P4	Widget shows yesterday after midnight; restore refreshes it before the transaction commits	Stale date and intake on the home screen for up to 30+ minutes after midnight.	Highly likely	app/src/main/res/xml/macrobase_widget_info.xml (updatePeriodMillis=1800000)
BUG-049	P4	Calendar and heatmap recolour past days against the current calorie goal	Lowering the goal from 2,000 to 1,600 turns past on-target days red.	Confirmed	feature/calendar/CalendarScreen.kt:152, 180, 200
BUG-050	P4	Values drift when users switch units or re-save unchanged entries	Small unexpected changes after unrelated edits.	Confirmed	feature/preferences/PreferencesScreen.kt:866-872
BUG-051	P4	Merge import treats custom foods as identical when only potassium, calcium or iron differ	An edited mineral value is skipped and counted as "skipped".	Confirmed	data/repository/PortabilityRepositoryImpl.kt:493-506
BUG-052	P4	Project-rule violations: String.format without Locale.US, raw non-ASCII literals, invalid UTF-8 in a scanner regex	Mixed decimal separators; micrograms written with the micro sign are not parsed.	Confirmed	feature/detail/FoodDetailScreen.kt:605, 626, 954, 1010
BUG-053	P4	DataStore read errors are not caught	Crash instead of falling back to defaults on a corrupt preferences file.	Highly likely	data/repository/GoalsRepositoryImpl.kt:222
BUG-054	P4	Search drops accented and non-Latin letters	No results for accented or Hindi queries (the catalog itself is romanised, so impact is limited).	Confirmed	data/provider/LocalFoodDatabaseProvider.kt:287-295 (sanitizeFtsQuery)
BUG-055	P4	Dead code and APK weight: unused PaddleOCR models, stub recent foods, misleading "Gemini" naming	Larger download; an advertised feature (recents) missing.	Confirmed	app/src/main/assets/models (about 30 MB)
BUG-056	P4	Scanner overlays let taps through, and bitmaps may be recycled while ML Kit still reads them	Overlapping scans or a rare native crash.	Potential	feature/scanner/NutritionLabelScannerScreen.kt:422-434
BUG-057	P4	Scanner header detection treats any element containing "100" or "serv" as a header	Lower-quality parses on some labels.	Confirmed	feature/scanner/SpatialTableDetector.kt:397-412

E. Detailed Findings

Findings are ordered by priority, then by number of users affected and severity. Paths are relative to app/src/main/java/com/macrobase/app/ unless they start with app/, tools/ or another top-level folder.

P1 BUG-008 The built APK declares INTERNET and ships ML Kit's Google telemetry transport, breaking the zero-network promise

Confidence: Confirmed (merged manifests and merger reports in app/build) **Category:** Security / Privacy

Area: Build / manifest

Location:

app/build/intermediates/merged_manifest/release/processReleaseMainManifest/AndroidManifest.xml:26-27, 132-145; app/build/outputs/logs/manifest-merger-release-report.txt:607-611; app/build.gradle.kts:97; app/src/main/AndroidManifest.xml

What is wrong: com.google.mlkit:text-recognition pulls in com.google.android.datatransport:transport-backend-cct, which merges INTERNET and ACCESS_NETWORK_STATE plus the TransportBackendDiscovery/JobInfoSchedulerService components (Firelog/CCT usage logging) into the app. The source manifest has no tools:node="remove" overrides.

How it happens: Build any variant; both the debug (8 Sep) and release (26 Aug) merged manifests contain the permissions.

What the user experiences: The app holds network permission and Google's logging transport while the UI says "Never uploaded to the cloud" (ImportExportScreen.kt:305) and AGENTS.md calls zero network non-negotiable.

Impact: Privacy claim is inaccurate; Play Data Safety answers based on it would be wrong; library usage metrics can leave the device.

Root cause: Transitive manifest merge was never audited.

Evidence:

```
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />
<uses-permission android:name="android.permission.INTERNET" />
...
uses-permission#android.permission.INTERNET
ADDED from [com.google.android.datatransport:transport-backend-cct:2.3.3]
```

Assumptions: The merged manifests are from builds on 26 Aug and 8 Sep; the dependency is unchanged since, so a fresh build should match. Exactly what ML Kit uploads was not verified (no network access in this audit).

Recommended fix: Add xmlns:tools and <uses-permission ... INTERNET tools:node="remove"/> (and ACCESS_NETWORK_STATE) to the app manifest; also remove the datatransport services/receivers, or exclude transport-backend-cct in Gradle. Re-verify OCR still works offline.

Regression test: Build-time check (Gradle task or CI script) that fails if any merged manifest contains android.permission.INTERNET.

Related findings: BUG-018

P2 BUG-011 Double-tapping Log or Save creates duplicate diary entries or pops past the dashboard

Confidence: Highly likely **Category:** Bug / Reliability **Area:** ViewModels + navigation

Location: feature/detail/FoodDetailScreen.kt:312-352 (logFood); feature/basket/BasketViewModel.kt:131-140 (submitBasket); domain/usecase/basket/BasketUseCases.kt:106-139 (CommitBasketUseCase)

What is wrong: Neither logFood nor submitBasket has an in-flight guard and the buttons are not disabled. CommitBasketUseCase reads items.value, suspends on the insert, and only then clears the basket, so a second tap re-reads the same items and inserts them again with fresh UUIDs.

How it happens: Tap "Log 3 Foods" twice quickly: up to 6 diary rows. Tap "Log 1 Food" in Food Detail twice: two rows. Double-tap Save/Delete in edit mode: two popBackStack calls.

What the user experiences: Duplicate foods on the dashboard and doubled totals; occasionally a blank screen after popping too far.

Impact: Inflated intake that users must find and delete by hand.

Root cause: Non-idempotent commit with no submission state.

Evidence:

```
fun submitBasket(onSuccess: () -> Unit, onError: (String) -> Unit) {
    viewModelScope.launch {
        val result = commitBasketUseCase()
        if (result.isSuccess) {
            onSuccess()
        }
    }
}
```

Assumptions: Tap timing: requires the second tap before the Room insert and navigation complete (tens of milliseconds); typical for double taps.

Recommended fix: Add isSubmitting to UI state, disable the buttons, take-and-clear the basket atomically under a Mutex, and navigate via a one-shot event.

Regression test: Call submitBasket twice concurrently with a slow fake repository; assert diary row count equals basket item count.

Related findings: BUG-015

P2 **BUG-012 Quantity fields accept blank, comma decimals, zero and negatives, logging 0 kcal or negative calories**

Confidence: Highly likely **Category:** Bug / Validation **Area:** UI input

Location: feature/detail/FoodDetailScreen.kt:266-282, 312-352; feature/recipes/RecipesScreen.kt:282, 138-141; feature/basket/BasketViewModel.kt:74-76

What is wrong: Food Detail parses with `toDoubleOrNull() ?: 0.0` and `logFood` has no validation. The recipe log dialog uses `toDoubleOrNull() ?: 1.0` and scales by the raw value, so "-1" produces negative nutrition. Comma decimals parse as null.

How it happens: On a German-locale keyboard the decimal key types "1,5": Food Detail logs quantity 0 and 0 kcal. Blank field: same. Recipe dialog "1,5" logs 1 serving; "-1" logs negative calories (once BUG-002 is fixed).

What the user experiences: Entries that add nothing, or that reduce the day's total.

Impact: Silent under-logging for comma-decimal locales.

Root cause: No input validation and locale-blind parsing.

Evidence:

```
val qty = newQuantityText.toDoubleOrNull() ?: 0.0
...
val qty = servingsCount.toDoubleOrNull() ?: 1.0
```

Assumptions: That the device keyboard emits a comma for `KeyboardType.Decimal` in the user's locale (common on Samsung/Gboard in de/fr).

Recommended fix: Normalise ',' to '.', reject `<= 0` and non-finite values, disable Log/Save while invalid, show an inline error.

Regression test: Parser cases: "1,5" -> 1.5; "" -> invalid; "-1" -> invalid; Log button disabled for invalid input.

Related findings: BUG-027, BUG-028

P2 **BUG-015 Recipe "Log to Diary" only adds to an in-memory basket, and lost basket items are reported as logged**

Confidence: Confirmed **Category:** UX / Reliability **Area:** Basket

Location: feature/recipes/RecipesScreen.kt:138-152, 296;
data/repository/basket/InMemoryBasketRepository.kt:15-17; feature/detail/FoodDetailScreen.kt:334-352;
domain/usecase/basket/BasketUseCases.kt:78-104

What is wrong: The recipe dialog's "Log to Diary" button calls addFoodToBasketUseCase and navigates home. The basket is a process-memory StateFlow. In Food Detail, commitSingleBasketItemUseCase returns success when the item is missing, and its Result is ignored before onSuccess().

How it happens: Log a recipe, background the app, let Android kill it: the recipe is gone. Or open a basket item, let the process die, return and tap Log: nothing is written, the screen still closes as if it succeeded.

What the user experiences: Meals the user believes are logged never reach the diary.

Impact: Silent data loss on low-memory devices.

Root cause: Volatile staging store; success reported without checking the outcome.

Evidence:

```
val item = items.firstOrNull { it.id == itemId } ?: return@runCatching
...
commitSingleBasketItemUseCase(currentBasketItemId)
...
_uiState.update { it.copy(isSavedSuccess = true) }
onSuccess()
```

Assumptions: Process death while the app is in the background (common on low-RAM devices).

Recommended fix: Persist the basket (Room table or DataStore); relabel the recipe button "Add to Basket" or commit directly; treat a missing item as an error and surface the Result.

Regression test: Commit a missing basket id: Result must be failure and onSuccess must not run.

Related findings: BUG-011, BUG-039

P2 BUG-016 Importing any backup schedules a reset of the fitness strategy to Maintaining / 2,000 kcal

Confidence: Confirmed **Category:** Data integrity **Area:** Backup / goals

Location: data/repository/PortabilityRepositoryImpl.kt:129-134 (export), 229-238 and 379-389 (import); data/repository/GoalsRepositoryImpl.kt:148-170; domain/model/Goal.kt:25-35

What is wrong: GoalBackupDto only carries calories and macro percentages. Import builds Goal(...) with those four fields, so fitnessGoal and maintenanceCalories take the defaults MAINTAINING and 2000.0. updateGoals sees a strategy change and schedules it for tomorrow with a transition record. Merge mode also overwrites goals and profile preferences unconditionally.

How it happens: User on Cutting with maintenance 2,600 imports a backup (Merge or Overwrite).

What the user experiences: Next day the strategy silently switches to Maintaining 2,000; goal history gains a fake transition; rank scoring changes.

Impact: Silent settings loss; strategy history and scheduled changes are never backed up.

Root cause: Partial goal DTO combined with default parameter values.

Evidence:

```
goalsRepository.updateGoals(  
    Goal(  
        dailyCalorieGoal = g.dailyCalorieGoal,  
        carbPercentage = g.carbPercentage,  
        proteinPercentage = g.proteinPercentage,  
        fatPercentage = g.fatPercentage  
    )  
)
```

Assumptions: None.

Recommended fix: Back up fitnessGoal, maintenanceCalories, scheduled fields and transition history; restore through a method that replaces goal state without scheduling. In Merge, keep local goals or ask.

Regression test: Fake repo on CUTTING/2600, import: getGoalForDate(tomorrow).fitnessGoal stays CUTTING and history is unchanged.

Related findings: BUG-024, BUG-031

P2 BUG-017 An archive missing a data file passes validation, and Overwrite restore then wipes that table

Confidence: Highly likely **Category:** Data loss / Reliability **Area:** Backup / import

Location: data/portability/BackupArchiveManager.kt:188-230;

feature/importexport/ImportExportScreen.kt:403-409; data/repository/PortabilityRepositoryImpl.kt:196-201

What is wrong: Checksums are only verified for files that exist; a listed file that is absent is skipped and parsed as an empty list. The preview shows manifest.counts, not the parsed sizes. Overwrite clears every table and inserts what was parsed.

How it happens: A truncated copy (ZipInputStream stops cleanly at an entry boundary), a hand-edited zip, or a partially written export lacking diary.json. Validation passes, preview says "Diary Entries 1500", user picks Overwrite.

What the user experiences: The diary is deleted; the result says "Successfully restored N records".

Impact: Total loss of the affected table with a success message.

Root cause: Missing files are not treated as corruption; counts never cross-checked.

Evidence:

```
val content = entryMap[filename]
if (content != null && expectedHash.isNotBlank()) {
    ...
} ?: emptyList()
```

Assumptions: An incomplete archive reaches the user (truncation, failed export left on disk, manual edit).

Recommended fix: Require every file listed in the manifest checksums; reject when parsed sizes differ from manifest.counts; show parsed counts in the preview; delete the SAF document if export fails.

Regression test: Archive without diary.json must be invalid; archive whose counts disagree must be invalid.

Related findings: BUG-006, BUG-047

P2 BUG-018 Android Auto Backup uploads the personal diary database to Google Drive despite the "never uploaded" claim

Confidence: Confirmed **Category:** Privacy **Area:** Manifest / backup rules

Location: app/src/main/AndroidManifest.xml:10-12; app/src/main/res/xml/backup_rules.xml:3;
app/src/main/res/xml/data_extraction_rules.xml:3-8; feature/importexport/ImportExportScreen.kt:305

What is wrong: allowBackup="true" and the rules only exclude built_in_foods.db. macrobase_user.db (diary, weight, custom foods) and the DataStore files (name, height, weight, goals) are included in cloud backup and device transfer.

How it happens: Any user with Google device backup enabled.

What the user experiences: The app states "Stored completely offline on your device. Never uploaded to the cloud." while Android uploads the data.

Impact: Misleading privacy statement; health data in a cloud account the user may not expect. Restoring that cloud backup onto an older app build also meets BUG-046.

Root cause: Default backup configuration kept.

Evidence:

```
android:allowBackup="true"
...
<exclude domain="database" path="built_in_foods.db" />
```

Assumptions: Device-level Google backup is enabled (default on many devices).

Recommended fix: Either exclude the user database and DataStore from <cloud-backup> (keep device-transfer if wanted) or set allowBackup="false"; or change the UI copy.

Regression test: Resource test parsing data_extraction_rules.xml asserting macrobase_user.db and datastore files are excluded from cloud-backup.

Related findings: BUG-008, BUG-046

P2 **BUG-019 Scanner: kilojoule energy values are stored as kilocalories (4.2x too high)**

Confidence: Confirmed (code traced, not executed) **Category:** Data integrity **Area:** Label scanner parser

Location: feature/scanner/SpatialTableDetector.kt:499-556; feature/scanner/NutritionNumericParser.kt:25-28; feature/scanner/NutritionLabelParser.kt:228-246, 299-305

What is wrong: The spatial path assigns the first two non-% tokens of a row to the value columns by order and never checks their unit. The calories cell is then normalised with sourceUnit = "kcal". The kJ-to-kcal conversion only runs when calories is null.

How it happens: EU label row "Energy 1046 kJ / 250 kcal | 523 kJ / 125 kcal": the first token 1046 becomes per-100 g calories. ANZ kJ-only label "Energy 680kJ 1700kJ": calories become 1700 kcal/100 g.

What the user experiences: A prefilled custom food with calories about 4.2x too high, marked as detected.

Impact: Wrong custom foods for users scanning EU/AU/NZ packaging; they must catch it during review.

Root cause: Unit-blind column assignment.

Evidence:

```
val p100 = NutritionNormalization.normalizeToPer100g(  
    sourceValue = cell100.rawToken.numericValue,  
    sourceUnit = "kcal",
```

Assumptions: Triggers when a calories row holds kJ values in the value columns (kJ and kcal on one row, or kJ-only tables). The draft is reviewed before saving and a macro-mismatch warning may appear, but the value is not corrected.

Recommended fix: Ignore or convert tokens whose unit is kJ in the calories row; assign tokens to the column whose X range contains them.

Regression test: Golden fixture "Energy 1046 kJ / 250 kcal | 523 kJ / 125 kcal" must yield 250 kcal per 100 g; extend test06 with a calories assertion.

Related findings: BUG-020, BUG-022, BUG-042

P2 BUG-020 Scanner: an unrecognised basis header defaults to "per 100 g" with high confidence

Confidence: Confirmed (code traced) **Category:** Data integrity **Area:** Label scanner parser

Location: feature/scanner/NutritionLabelParser.kt:692-717 (detectBasis)

What is wrong: If the text lacks "per 100 g", "per serving", "per serve", "per portion" or "serving size", the basis silently becomes PER_100_G and the first value column is trusted as per-100 g without needing review.

How it happens: Canadian label "Per 1 cup (250 mL) / Calories 110" or UK "Per bar (40g)": 110 kcal is saved as per 100 g.

What the user experiences: Per-serving numbers stored as per-100 g with no warning; the macro check stays quiet because the numbers are self-consistent.

Impact: Custom foods off by the serving weight factor.

Root cause: Optimistic default instead of UNKNOWN.

Evidence:

```
return NutritionBasis.PER_100_G // Standard default food basis
```

Assumptions: User accepts the prefilled form.

Recommended fix: Default to UNKNOWN (forces review); recognise "Per <qty> <unit> (<n> g|mL)" and "Per bar/pack/piece" headers as per serving and capture grams.

Regression test: parseText on the Canadian sample must not return PER_100_G with HIGH confidence.

Related findings: BUG-019, BUG-021

P2 **BUG-021 Scanner: per-serving values without a gram weight are relabelled as per 100 g**

Confidence: Confirmed **Category:** Data integrity **Area:** Scanner model

Location: domain/model/scanner/ScannerModels.kt:149-166 (withOutputBasis);
feature/scanner/NutritionLabelScannerScreen.kt (review modal basis options)

What is wrong: When normalisation to per-100 g was refused (no serving grams), to100g falls back to the raw per-serving value and sets basis PER_100_G, servingSize 100 g.

How it happens: "Serving size 1 bar / Calories 190 / Protein 15g": the modal defaults to Per 100 g showing 190; Apply yields a food "100 g = 190 kcal". Logging the 40 g bar then records 76 kcal.

What the user experiences: A warning asks for the serving weight, but the modal has no field for it; the saved food is wrong.

Impact: Under- or over-reporting for every scanned label without a gram weight.

Root cause: Silent fallback in a unit conversion.

Evidence:

```
val fromMap = per100gValues[key]
if (fromMap != null) return fromMap
if (primary == null) return null
val val100g = primary.normalizedPer100g ?: primary.value
```

Assumptions: None.

Recommended fix: Disable Per 100 g / Per 1 g output when servingGrams is unknown and save as 1 serving, or add a grams input to the modal.

Regression test: Extend golden test20: withOutputBasis(PER_100_G) must not return calories 190 with servingSize 100 g.

Related findings: BUG-020

P2 **BUG-022 Scanner: thousands separators are read as decimals ("1,580 mg" becomes 1.58 mg)**

Confidence: Confirmed **Category:** Data integrity **Area:** Label scanner parser

Location: feature/scanner/NutritionLabelParser.kt:1033-1042;

feature/scanner/NutritionNumericParser.kt:40; feature/scanner/NutritionDigitCorrector.kt:14
(parseDecimal, never called)

What is wrong: Both parsing paths replace every comma with a dot. NutritionDigitCorrector.parseDecimal, which handles thousands separators, is not used anywhere.

How it happens: "Sodium 1,580mg" gives 1.58 mg; "Energy 1,046 kJ" (kJ only) gives about 0.25 kcal.

What the user experiences: Sodium or energy about 1,000x too low with no review flag.

Impact: Dangerous for users tracking sodium (hypertension); wrong energy for kJ labels.

Root cause: Naïve decimal normalisation.

Evidence:

```
val normalizedText = normalizeOcrMisreads(element.text).replace(',', ' ')
...
val numStr = match.groupValues[2].trim().replace(',', ' ')
```

Assumptions: None.

Recommended fix: Route all numeric parsing through parseDecimal and accept `\d{1,3}(\d{3})+` patterns.

Regression test: `parseText("Serving size 1 cup (240g)\nSodium 1,580mg 69%")` must give about 658 mg per 100 g.

Related findings: BUG-019

P2 **BUG-023 Rank rating can never exceed 600 of 2,399, so 5 of the 8 ranks are unreachable**

Confidence: Confirmed **Category:** Bug **Area:** Rank feature

Location: data/repository/rank/RankRepositoryImpl.kt:39-41, 161-218;
core/config/RankScoringConfig.kt:21, 28, 36

What is wrong: The rating is rebuilt from 0 on every emission using only the last 30 days, with at most +20 per day.

How it happens: Perfect logging every day yields at most $30 \times 20 = 600$, which maps to the third rank. Good days falling out of the window lower the rating even with perfect behaviour.

What the user experiences: The rank ladder stalls permanently at rank 3; ratings drop for no visible reason.

Impact: The gamification feature cannot deliver its advertised progression; tests (test6_maximumCeiling) assume higher ranks are reachable.

Root cause: No persisted rating; a rolling window is used as if it were full history.

Evidence:

```
val startDate = today.minusDays((RankScoringConfig.ROLLING_WINDOW_DAYS - 1).toLong())
...
var runningRating = RankScoringConfig.MIN_RATING
...
runningRating = (runningRating + rrDelta).coerceIn(RankScoringConfig.MIN_RATING,
RankScoringConfig.MAX_RATING)
```

Assumptions: None.

Recommended fix: Persist rating plus last evaluated date and apply deltas for newly completed days, or replay over the full history.

Regression test: Fake 120 perfect days: rating must exceed 600.

Related findings: BUG-024, BUG-032

P2 BUG-024 Rank scores days against stale calorie and macro targets

Confidence: Confirmed **Category:** Bug / Data integrity **Area:** Goals repository

Location: data/repository/GoalsRepositoryImpl.kt:148-170, 176-187, 242-266 (getGoalForDate);
data/repository/rank/RankRepositoryImpl.kt:172-192

What is wrong: Transition records are written only when strategy or maintenance changes. Calorie and macro edits overwrite the live values but not the history, and getGoalForDate returns the stored record's targets. The baseline record is also built after the new values were written, so earlier days are re-scored with the new targets.

How it happens: Switch to Cutting at 2,000 kcal 50/25/25; later change only to 2,200 kcal 30/40/30. Every day since the switch is scored against a 125 g protein target instead of 220 g; eating the displayed target scores poorly and blocks positive RR.

What the user experiences: Following the on-screen targets is penalised; the rank breakdown disagrees with the dashboard.

Impact: Rank results are wrong for anyone who adjusts macros.

Root cause: History only tracks part of the goal state.

Evidence:

```
prefs[Keys.DAILY_CALORIE_GOAL] = goals.dailyCalorieGoal
...
val strategyChanged = (goals.fitnessGoal != currentFitnessGoal) || (goals.maintenanceCalories !=
currentMaintenance)
```

Assumptions: None.

Recommended fix: Capture old values before overwriting and append a transition record whenever any goal field changes (effective today).

Regression test: Change strategy, then macros only: getGoalForDate(today).proteinGrams equals observeGoals().first().proteinGrams.

Related findings: BUG-023, BUG-016, BUG-049

P3 BUG-026 Macro-percentage check differs between UI and use case, so some valid goal saves fail silently

Confidence: Confirmed **Category:** Bug **Area:** Goals

Location: domain/model/Goal.kt:56-67; domain/usecase/BiometricAndGoalUseCases.kt:139;
feature/preferences/PreferencesScreen.kt:129-133

What is wrong: The UI validates with totalPercentageExact, but UpdateGoalsUseCase uses isValidPercentageSum, which truncates the sum with toInt(). saveAll ignores the failure.

How it happens: Carbs 30.4, protein 34.8, fat 34.8 sums to 99.99999999999999 in floating point; truncation gives 99, the use case fails. A 33.5 x 3 split (100.5) passes.

What the user experiences: Save appears to work; preferences are stored but goals are not, with no message.

Impact: Users believe new targets are active when they are not.

Root cause: Truncating comparison and ignored Result.

Evidence:

```
val totalPercentage: Int
    get() = (carbPercentage + proteinPercentage + fatPercentage).toInt()
val isValidPercentageSum: Boolean
    get() = abs(totalPercentage - 100.0) < 0.01
...
if (goalRes.isSuccess) { onSuccess() }
```

Assumptions: None.

Recommended fix: Validate with `abs(totalPercentageExact - 100.0) < 0.01` and show the failure message.

Regression test: Goal(30.4, 34.8, 34.8).isValidPercentageSum is true; Goal(33.5, 33.5, 33.5) is false.

Related findings: BUG-028

P3 BUG-027 Weight edit dialog rejects its own pre-filled value in comma-decimal locales

Confidence: Confirmed **Category:** Bug / Localization **Area:** Weight screen

Location: feature/weight/WeightScreen.kt:650-656, 724-731, 771-774

What is wrong: Pre-fill uses `String.format("%.1f")` with the default locale ("72,5" in de-DE), and Save parses with `toDoubleOrNull()`.

How it happens: On a German or French device open Edit on a 72.5 kg entry and tap Save without changes.

What the user experiences: "Please enter a valid positive number"; typed "72,5" is also rejected.

Impact: Weight cannot be edited in comma locales.

Root cause: Locale-dependent formatting paired with locale-blind parsing (AGENTS.md 3.1).

Evidence:

```
String.format("%.1f", UnitConversions.kgToLbs(entry.weightKg))
...
val parsed = editInput.toDoubleOrNull()
```

Assumptions: None.

Recommended fix: Format with `Locale.US` and accept ',' when parsing.

Regression test: With `Locale.GERMANY` as default, prefill then parse round-trips 72.5.

Related findings: BUG-012, BUG-052

P3 BUG-028 Preferences inputs are not validated; comma or cleared values are silently dropped

Confidence: Confirmed **Category:** Bug / Validation **Area:** Preferences

Location: feature/preferences/PreferencesScreen.kt:855-872;
data/repository/PreferencesRepositoryImpl.kt:57-61

What is wrong: Weight, height and target weight parse with `toDoubleOrNull()`; null values are skipped on write so the old value survives; water goal falls back to 2,500; negative or zero values are accepted.

How it happens: Type "72,5" for weight, clear target weight, or enter -100 for water and save.

What the user experiences: "Saved" message, but weight unchanged, cleared target reappears, or a negative water goal is stored.

Impact: Profile values drift from what the user entered.

Root cause: No validation; null used as "unchanged".

Evidence:

```
val rawWater = waterGoalInput.toDoubleOrNull() ?: 2500.0
...
preferences.currentWeightKg?.let { prefs[Keys.CURRENT_WEIGHT_KG] = it }
```

Assumptions: None.

Recommended fix: Locale-tolerant parsing, range checks gating Save, and explicit removal when a field is cleared.

Regression test: ViewModel test for clearing a value and rejecting "-5" and "72,5" handling.

Related findings: BUG-027, BUG-029

P3 BUG-029 Toggling the unit system discards typed preference edits

Confidence: Confirmed **Category:** Bug / UX **Area:** Preferences

Location: feature/preferences/PreferencesScreen.kt:150-182

What is wrong: Input fields are `remember(prefs, selectedUnitSystem) { ...from stored prefs... }`, so switching units re-derives them from saved values.

How it happens: Stored weight 75 kg; type 80; toggle to Imperial: the field shows 165.3 lb (75 kg); Save stores 75.

What the user experiences: The typed value vanishes; strategy auto-detection keeps the value from 80, so the two disagree.

Impact: Lost input.

Root cause: State keyed on the unit toggle.

Evidence:

```
var weightInput by remember(prefs, selectedUnitSystem) {  
    val kg = prefs.currentWeightKg
```

Assumptions: None.

Recommended fix: Convert the currently typed text when the unit changes.

Regression test: Compose test: type, toggle, assert converted typed value.

Related findings: BUG-028, BUG-050

P3 BUG-030 Tapping the home-screen widget never opens Search and resets the app's back stack

Confidence: Confirmed **Category:** Bug **Area:** Widget / MainActivity

Location: MainActivity.kt:21-41; feature/widget/MacroBaseWidgetProvider.kt:123-124

What is wrong: The widget intent uses `NEW_TASK | CLEAR_TOP` with standard launch mode, so MainActivity is recreated and `onCreate` runs. `onCreate` calls `tryEmit` on a `MutableSharedFlow` with `replay 0` before `setContent` subscribes; with no subscribers the value is dropped.

How it happens: Tap the widget (app running or not).

What the user experiences: The dashboard opens instead of Search; any screen the user had open is gone.

Impact: The widget's only action does not work.

Root cause: Event emitted before a collector exists.

Evidence:

```
private val widgetActionFlow = MutableSharedFlow<String?>(extraBufferCapacity = 1)  
...  
if (intent?.action == MacroBaseWidgetProvider.ACTION_WIDGET_LOG_FOOD) {  
    widgetActionFlow.tryEmit(intent?.action)  
}
```

Assumptions: None.

Recommended fix: Use a `Channel` or `replay = 1` (consume after handling), or read `intent.action` inside a `LaunchedEffect`; add `FLAG_ACTIVITY_SINGLE_TOP` or `launchMode="singleTop"`.

Regression test: Instrumented test launching with `ACTION_WIDGET_LOG_FOOD` asserts the Search route.

Related findings: BUG-048

P3 BUG-031 A pending fitness-strategy change cannot be cancelled

Confidence: Confirmed **Category:** Bug **Area:** Goals / Preferences

Location: feature/preferences/PreferencesScreen.kt:189; data/repository/GoalsRepositoryImpl.kt:164, 206-213

What is wrong: The selector starts from the active strategy and updateGoals compares against the active strategy, so reverting to it is "no change" and the scheduled keys and tomorrow's transition stay.

How it happens: Maintaining -> Cutting (scheduled for tomorrow) -> back to Maintaining, Save.

What the user experiences: Tomorrow Cutting is applied anyway.

Impact: User intent ignored.

Root cause: Comparison ignores the pending state.

Evidence:

```
var selectedGoal by remember(goals) { mutableStateOf(goals.fitnessGoal) }
```

Assumptions: None.

Recommended fix: Compare against the pending value; clear SCHEDULED_* and tomorrow's record when the selection equals the active strategy.

Regression test: Schedule, revert, advance one day: strategy is still Maintaining.

Related findings: BUG-016

P3 BUG-032 Rank penalises today before the day is over (up to -20 RR every morning)

Confidence: Confirmed (behaviour); design intent unclear **Category:** Bug / UX **Area:** Rank feature

Location: data/repository/rank/RankRepositoryImpl.kt:165-218;
data/repository/CoreRepositoryImplementations.kt:294-295

What is wrong: The evaluation loop includes today, so an unlogged or partly logged day is scored as complete. today is also captured once when the flow is built.

How it happens: Open Rank before breakfast: lifestyle score about 4, delta clamped to -20.

What the user experiences: Rank drops every morning and recovers during the day.

Impact: Confusing, demotivating feedback.

Root cause: Partial day treated as final.

Evidence:

```
var curr = startDate  
while (!curr.isAfter(today)) {
```

Assumptions: None.

Recommended fix: Evaluate through yesterday; show today as a provisional preview not added to the rating.

Regression test: With no entries today, today's delta is excluded from the rating.

Related findings: BUG-023

P3 BUG-033 Statistics windows include one extra day and the consistency score is truncated

Confidence: Confirmed **Category:** Bug **Area:** Statistics

Location: feature/statistics/StatisticsScreen.kt:144-149, 195-200;
data/repository/CoreRepositoryImplementations.kt:294-297

What is wrong: "30 Days" uses today.minusDays(30) inclusive (31 days) and the score uses toInt().

How it happens: Log each of the last 30 days but not day 31.

What the user experiences: "96%" instead of 100%; 7-day averages span 8 days.

Impact: Misreported adherence.

Root cause: Off-by-one range and truncation.

Evidence:

```
val cFrom = today.minusDays(cInt.days)
...
((loggedDays.toDouble() / eligibleCells.size.toDouble()) * 100.0).toInt()
```

Assumptions: None.

Recommended fix: Use minusDays(days - 1) and roundToInt().

Regression test: Exactly 30 logged days gives 30 eligible days and 100%.

Related findings: BUG-034

P3 BUG-037 Missing fiber, sugar and sodium are stored as 0.0 instead of null in diary snapshots

Confidence: Confirmed **Category:** Data integrity / AGENTS.md 2.4 **Area:** Repository + schema

Location: data/database/entity/UserEntities.kt:29-31; data/repository/DiaryRepositoryImpl.kt:108-110, 134-136, 159-161

What is wrong: Columns are NOT NULL with default 0.0 and the mapper writes ?: 0.0, so "unknown" and "zero" become the same value. Saturated fat, trans fat and cholesterol are not snapshotted at all.

How it happens: Log a food with no fiber data.

What the user experiences: History shows "0.0 g" where it should show "-"; this 0 is what triggers BUG-010.

Impact: Misleading micronutrient history.

Root cause: Schema chose non-null defaults.

Evidence:

```
loggedFiber = entry.calculatedNutrition.fiberGrams ?: 0.0,
loggedSugar = entry.calculatedNutrition.sugarGrams ?: 0.0,
loggedSodium = entry.calculatedNutrition.sodiumMg ?: 0.0,
```

Assumptions: None.

Recommended fix: Add nullable columns via a non-destructive migration (or a sentinel) and map null through.

Regression test: Log a food with null fiber; reloaded entry has fiberGrams == null.

Related findings: BUG-010, BUG-007

P3 BUG-038 Food Detail resets the typed quantity and serving on rotation

Confidence: Highly likely **Category:** UX **Area:** Food Detail

Location: feature/detail/FoodDetailScreen.kt:428-430

What is wrong: LaunchedEffect(foodId, entryId, basketItemId) runs again when the activity is recreated and loadFood overwrites the retained ViewModel state; the manifest declares no configChanges.

How it happens: Type 2.5, rotate the phone.

What the user experiences: Quantity returns to the default (or to the stored entry in edit mode).

Impact: Lost input; possible wrong log if unnoticed.

Root cause: Unconditional reload.

Evidence:

```
LaunchedEffect(foodId, entryId, basketItemId) {
    viewModel.loadFood(foodId, mealType, date, entryId, basketItemId)
}
```

Assumptions: None.

Recommended fix: Skip loadFood when the ViewModel already holds these arguments; use SavedStateHandle.

Regression test: Recreate the activity in a test and assert the typed quantity survives.

Related findings: None

P3 BUG-039 Basket: errors are swallowed and the header shows only the first item's date and meal

Confidence: Confirmed **Category:** UX / Reliability **Area:** Basket

Location: feature/basket/BasketScreen.kt:518-524, 283-296; feature/basket/BasketViewModel.kt:56-70

What is wrong: onError is an empty lambda; the "When:" header mirrors the first item while items keep their own dates/meals; the "Single Food (Recipe)" option is stored but never used.

How it happens: A commit failure, or a basket with mixed dates.

What the user experiences: Nothing happens on failure; items log to dates the header did not show.

Impact: Confusing, silent failures.

Root cause: Unfinished UI wiring.

Evidence:

```
viewModel.submitBasket(  
    onSuccess = onNavigateToDashboard,  
    onError = { /* show error */ }  
)
```

Assumptions: None.

Recommended fix: Show a snackbar on error; show per-item date/meal when they differ; remove or implement the recipe mode.

Regression test: Failing commit shows an error message.

Related findings: BUG-015

P3 BUG-040 Dashboard keeps yesterday's date after midnight, so new logs go to the wrong day

Confidence: Potential **Category:** Bug **Area:** Dashboard

Location: feature/dashboard/HomeScreen.kt:108

What is wrong: _selectedDate is set once when the activity-scoped ViewModel is created and only reset by navigation to Home.

How it happens: Leave the app open overnight, then tap + in the morning.

What the user experiences: Entries land on yesterday; the header changes from "Today" to a date.

Impact: Misfiled meals.

Root cause: No day-rollover handling.

Evidence:

```
private val _selectedDate = MutableStateFlow(LocalDate.now(ZoneId.systemDefault()))
```

Assumptions: The activity is not destroyed overnight (common when the app stays in recents).

Recommended fix: On resume, move the selection to today if it previously was today.

Regression test: Advance a fake clock past midnight; selection follows.

Related findings: BUG-048

P3 **BUG-041 Scanner: with a single per-serving column, a second number in the row becomes the per-100 g value**

Confidence: Highly likely **Category:** Data integrity **Area:** Label scanner parser

Location: feature/scanner/NutritionLabelParser.kt:1105-1121

What is wrong: In the single PER_SERVING branch, any second non-% token is assigned to per100g, which then outranks the per-serving value.

How it happens: US label "Sodium 160mg 7 %" OCR'd as "160mg", "7", "%": per-100 g sodium becomes 7.

What the user experiences: Wrong sodium with only a "Check" badge.

Impact: Wrong micronutrients on US-style labels.

Root cause: Extra token treated as a second column.

Evidence:

```
perServing = valueTokens[0].value
...
if (valueTokens.size >= 2) {
    per100g = valueTokens[1].value
```

Assumptions: OCR splits the % sign into its own element.

Recommended fix: Never fill per100g from extra tokens when only a per-serving column exists; ignore numbers followed by a separate "%".

Regression test: Fixture "Sodium 160mg 7 %" yields per-serving 160.

Related findings: BUG-019

P3 BUG-042 Scanner: adjudicator warnings are discarded and critical warnings can be hidden

Confidence: Confirmed **Category:** UX / Data integrity **Area:** Scanner orchestration + UI

Location: feature/scanner/NutritionLabelScanOrchestrator.kt:214-245;
feature/scanner/NutritionLabelScannerScreen.kt:1190

What is wrong: combinedWarnings is captured before adjudication, so the adjudicator's range and discrepancy warnings never reach the UI; the range check only runs for low-confidence drafts; the modal shows only the first three warnings, with image-quality hints first.

How it happens: A 1,700 kcal/100 g kJ misread (BUG-019) on a slightly blurry photo.

What the user experiences: No visible "calories too high" warning.

Impact: Safety net does not work.

Root cause: Ordering bug and truncated list.

Evidence:

```
val combinedWarnings = quality.warnings + draft.warnings
if (adjudicator.shouldAdjudicate(draft)) { ... draft = adjudicator.adjudicate(draft, tableGrid) }
...
for (warn in draft.warnings.take(3)) {
```

Assumptions: None.

Recommended fix: Build warnings after adjudication, always run range checks, sort critical warnings first or show all.

Regression test: Orchestrator test with fake OCR: a 1,700 kcal/100 g draft has a range warning in the first three.

Related findings: BUG-019

P3 BUG-043 Scanner: table-crop OCR coordinates are mixed with full-image coordinates

Confidence: Highly likely **Category:** Data integrity **Area:** Scanner orchestration

Location: feature/scanner/NutritionLabelScanOrchestrator.kt:159-175; feature/scanner/OcrEnsemble.kt:150-172

What is wrong: The TableCrop pass OCRs a cropped bitmap, but its boxes are never offset back; the ensemble merges lines by Y distance alone.

How it happens: Labels framed with margin above the table.

What the user experiences: Intermittently swapped or duplicated nutrient rows.

Impact: Hard-to-reproduce wrong values.

Root cause: Missing coordinate translation.

Evidence:

```
val cropOcr = ocrEngine.recognizeText(tableCrop)
...
abs(lineY - exY) < 20
```

Assumptions: None.

Recommended fix: Translate crop-pass boxes by the crop origin before scoring and merging.

Regression test: buildEnsemble with an offset crop pass yields one coordinate space and no duplicate rows.

Related findings: BUG-019

P3 BUG-044 Scanner: gallery images are decoded at full size on the main thread and can crash with OutOfMemoryError

Confidence: Confirmed (code); crash depends on image size **Category:** Reliability / Performance **Area:** Scanner screen

Location: feature/scanner/NutritionLabelScannerScreen.kt:107-122

What is wrong: BitmapFactory.decodeStream runs on the main thread without inSampleSize; catch (e: Exception) does not catch OutOfMemoryError. The picked image also bypasses quality checks (isTestImage = true).

How it happens: Pick a 50 MP photo from the gallery.

What the user experiences: Freeze, then app crash.

Impact: Crash on a user-reachable button.

Root cause: Unbounded decode on the UI thread.

Evidence:

```
val bitmap = context.contentResolver.openInputStream(uri)?.use { stream ->
    BitmapFactory.decodeStream(stream)
}
```

Assumptions: The gallery button is visible in production builds (no build-type gate was found).

Recommended fix: Decode off the main thread with bounds probing and sampling; catch Throwable; keep quality checks.

Regression test: Decode a 8000x6000 image under a constrained heap without crashing.

Related findings: None

P3 BUG-045 Scanner: "Calories from Fat" rows take over the fat and calorie values on older US labels

Confidence: Highly likely **Category:** Data integrity **Area:** Label scanner parser

Location: feature/scanner/NutritionLabelParser.kt:945-953, 1014-1029

What is wrong: The "calories from fat" row is excluded only for energy; for fat, the first row matching \bfat\b wins.

How it happens: Row "Calories 230 Calories from Fat 72" above "Total Fat 8g".

What the user experiences: Fat of 72 g/100 g; calories may fall to the footnote "2,000" (then BUG-022).

Impact: Wrong macros on legacy labels.

Root cause: Keyword match not anchored to the row label.

Evidence:

```
if (text.contains("calories from fat") || text.contains("cal from fat")) false
```

Assumptions: None.

Recommended fix: Skip "calories from fat" rows for every nutrient; require the keyword at the start of the row.

Regression test: Legacy US fixture yields fat 8 g and calories 230.

Related findings: BUG-022

P3 BUG-046 Destructive migration fallback wipes all user data on a downgrade or a missed migration

Confidence: Potential **Category:** Data loss / Reliability **Area:** Database setup

Location: core/di/DiModules.kt:91-99; data/database/UserDatabase.kt:19-29

What is wrong: .fallbackToDestructiveMigration() deletes and recreates every table whenever Room finds no migration path, including downgrades. exportSchema = false means there is no schema history to catch a forgotten migration.

How it happens: Install an older APK over a newer one, restore a cloud backup (BUG-018) into an older build, or ship a schema change without a migration.

What the user experiences: All diary, custom foods, recipes, weight and water history disappear on launch, with no warning.

Impact: Catastrophic for the affected user, though the trigger is uncommon.

Root cause: Convenience setting left on for production.

Evidence:

```
.addMigrations(UserDatabase.MIGRATION_1_2, UserDatabase.MIGRATION_2_3)
.fallbackToDestructiveMigration()
```

Assumptions: A downgrade or forgotten migration occurs; current 1->2->3 paths are present.

Recommended fix: Remove the fallback (or limit it to debug), enable exportSchema and add MigrationTestHelper tests for every version.

Regression test: Migration test from v1 and v2 schemas to v3 preserving rows.

Related findings: BUG-018

P3 BUG-047 A crafted or very large backup can crash the app; parse errors embed the whole file in the UI

Confidence: Potential **Category:** Reliability / Security **Area:** Backup / import

Location: data/portability/BackupArchiveManager.kt:24-28; data/portability/BackupJsonSerializer.kt:400-412

What is wrong: Limits allow 100 MB of uncompressed JSON held as byte arrays and strings, then parsed into an object graph; OutOfMemoryError is not caught. Error messages include the entire JSON text.

How it happens: Import a 100 KB zip that expands to about 100 MB, or a large malformed diary.json.

What the user experiences: App crash, or a huge error string rendered in a Text (jank/ANR).

Impact: Crash on import of untrusted files.

Root cause: Generous limits and verbose errors.

Evidence:

```
const val MAX_TOTAL_UNCOMPRESSED_BYTES = 100 * 1024 * 1024L
...
throw IllegalArgumentException("JSON is not an object: $json")
```

Assumptions: A hostile or unusually large archive.

Recommended fix: Lower limits (for example 20 MB), stream-parse, catch OutOfMemoryError, and cap error text to position plus a short snippet.

Regression test: Extraction of a 90 MB-expansion zip fails gracefully; error messages stay under 300 characters.

Related findings: BUG-017

P4 BUG-048 Widget shows yesterday after midnight; restore refreshes it before the transaction commits

Confidence: Highly likely **Category:** Reliability **Area:** Widget

Location: app/src/main/res/xml/macrobse_widget_info.xml (updatePeriodMillis=1800000); data/repository/PortabilityRepositoryImpl.kt:196-238

What is wrong: No receiver for DATE_CHANGED/TIME_SET/TIMEZONE_CHANGED, so the widget refreshes only every 30 minutes or later under Doze. Restore triggers widget updates through updateGoals, which can read pre-commit data.

How it happens: See "What is wrong".

What the user experiences: Stale date and intake on the home screen for up to 30+ minutes after midnight.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
android:updatePeriodMillis="1800000"
```

Assumptions: None.

Recommended fix: Refresh on date/time-change broadcasts or a midnight alarm; refresh after the import transaction.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-030, BUG-040

P4 BUG-049 Calendar and heatmap recolour past days against the current calorie goal

Confidence: Confirmed (behaviour) **Category:** UX / Design **Area:** Calendar

Location: feature/calendar/CalendarScreen.kt:152, 180, 200; data/repository/DiaryRepositoryImpl.kt:394-421

What is wrong: Every day is compared with goals.dailyCalorieGoal instead of the goal in force on that date.

How it happens: See "What is wrong".

What the user experiences: Lowering the goal from 2,000 to 1,600 turns past on-target days red.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
calorieGoal = goals.dailyCalorieGoal,
```

Assumptions: None.

Recommended fix: Use getGoalForDate once BUG-024 is fixed.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-024

P4 BUG-050 Values drift when users switch units or re-save unchanged entries

Confidence: Confirmed **Category:** Data integrity (minor) **Area:** Preferences / Water / Weight

Location: feature/preferences/PreferencesScreen.kt:866-872; feature/water/WaterScreen.kt:549-556; feature/weight/WeightScreen.kt:724-731

What is wrong: Imperial pre-fills round to whole fl oz or one decimal lb and saving converts back, so 2,500 mL becomes 2,513.75 mL and a 250 mL entry becomes 236.6 mL.

How it happens: See "What is wrong".

What the user experiences: Small unexpected changes after unrelated edits.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
UnitConversions.mlToFlOz(entry.amountMl).roundToInt().toString()
```

Assumptions: None.

Recommended fix: Track dirty fields and only write what changed; use finer precision for fl oz.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-029

P4 **BUG-051 Merge import treats custom foods as identical when only potassium, calcium or iron differ**

Confidence: Confirmed **Category:** Data integrity (minor) **Area:** Backup / import

Location: data/repository/PortabilityRepositoryImpl.kt:493-506

What is wrong: isIdenticalTo stops at sodiumMg.

How it happens: See "What is wrong".

What the user experiences: An edited mineral value is skipped and counted as "skipped".

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
sugarGrams == dto.sugarGrams &&  
sodiumMg == dto.sodiumMg
```

Assumptions: None.

Recommended fix: Compare every field.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-006

P4 **BUG-052 Project-rule violations: String.format without Locale.US, raw non-ASCII literals, invalid UTF-8 in a scanner regex**

Confidence: Confirmed **Category:** Localization / Maintainability **Area:** Many files

Location: feature/detail/FoodDetailScreen.kt:605, 626, 954, 1010; feature/recipes/RecipesScreen.kt:340, 360-362, 481-483, 532; feature/weight/WeightScreen.kt:469, 654, 727, 729, 882; feature/statistics/StatisticsScreen.kt:424, 766-774; feature/scanner/NutritionNumericParser.kt:26; data/portability/BackupArchiveManager.kt:284

What is wrong: AGENTS.md 3.1/3.2 are broken in many places. Most are cosmetic ("12,5 g" in de-DE); WeightScreen ones cause BUG-027. NutritionNumericParser.kt:26 contains raw 0xB5 bytes that are not valid UTF-8, so the "\u00b5g" alternative becomes U+FFFD and never matches.

How it happens: See "What is wrong".

What the user experiences: Mixed decimal separators; micrograms written with the micro sign are not parsed.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
// RecipesScreen.kt:360
Text(text = "P: ${String.format("%.1f", perServing.proteinGrams)}g", ...)
// WeightScreen.kt:727
String.format("%.1f", UnitConversions.kgToLbs(entry.weightKg))
// RecipesScreen.kt:340 contains a raw U+2022 bullet literal between "servings" and the ingredient
count
```

Assumptions: None.

Recommended fix: Pass Locale.US everywhere; replace literals with \u escapes; re-save NutritionNumericParser.kt as UTF-8 with "\u00b5\u03bc".

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-027

P4 BUG-053 DataStore read errors are not caught

Confidence: Highly likely **Category:** Reliability **Area:** Goals / Preferences repositories

Location: data/repository/GoalsRepositoryImpl.kt:222; data/repository/PreferencesRepositoryImpl.kt

What is wrong: No `.catch { IOException }` on DataStore data flows, so a read or corruption error crashes collectors.

How it happens: See "What is wrong".

What the user experiences: Crash instead of falling back to defaults on a corrupt preferences file.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
val prefs = context.goalsDataStore.data.first()
```

Assumptions: None.

Recommended fix: Catch `IOException` and emit `emptyPreferences()`; add a corruption handler.

Regression test: Add a unit or lint check for the specific case described.

Related findings: None

P4 BUG-054 Search drops accented and non-Latin letters

Confidence: Confirmed **Category:** UX **Area:** Food search

Location: data/provider/LocalFoodDatabaseProvider.kt:287-295 (`sanitizeFtsQuery`)

What is wrong: Every character outside `[a-zA-Z0-9 ]` is replaced with a space, so `"jalape\u00f1o"` becomes `"jalape* o*"` and Devanagari input becomes empty, returning no results without trying the LIKE fallback.

How it happens: See "What is wrong".

What the user experiences: No results for accented or Hindi queries (the catalog itself is romanised, so impact is limited).

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
val tokens = raw.replace(Regex("[^a-zA-Z0-9 ]"), " ")
```

Assumptions: None.

Recommended fix: Keep Unicode letters (`\p{L}\p{N}`); the FTS table already uses `unicode61` with `remove_diacritics`.

Regression test: Add a unit or lint check for the specific case described.

Related findings: None

P4 **BUG-055 Dead code and APK weight: unused PaddleOCR models, stub recent foods, misleading "Gemini" naming**

Confidence: Confirmed **Category:** Maintainability / Performance **Area:** Scanner, search

Location: app/src/main/assets/models (about 30 MB); feature/scanner/PaddleOcrEngine.kt:15 (never instantiated); data/repository/FoodRepositoryImpl.kt:78-80;
feature/scanner/NutritionLabelGeminiAdjudicator.kt

What is wrong: About 30 MB of ONNX models plus onnxruntime/OpenCV ship but PaddleOcrEngine is never created. getRecentFoods returns an empty list, so "Recently logged" never appears. The Gemini-named adjudicator contains no network code but has unused apiKey parameters, which invites someone to add a cloud call.

How it happens: See "What is wrong".

What the user experiences: Larger download; an advertised feature (recents) missing.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
override suspend fun getRecentFoods(limit: Int): List<Food> {  
    return emptyList()  
}
```

Assumptions: None.

Recommended fix: Remove unused models/libraries or wire them in; implement recents from diary_entries; rename the adjudicator and drop apiKey.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-008

P4 BUG-056 Scanner overlays let taps through, and bitmaps may be recycled while ML Kit still reads them

Confidence: Potential **Category:** Reliability **Area:** Scanner screen / orchestrator

Location: feature/scanner/NutritionLabelScannerScreen.kt:422-434;
feature/scanner/NutritionLabelScanOrchestrator.kt:125-131

What is wrong: The analyzing overlay is a plain Box with no pointer handling, so the gallery/back/torch buttons stay tappable; on cancellation the finally block recycles variant bitmaps that an in-flight recognizer task may reference.

How it happens: See "What is wrong".

What the user experiences: Overlapping scans or a rare native crash.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
for (v in variants) {  
    if (v.bitmap != safeBitmap && v.bitmap != bitmap) {  
        v.recycleIfNeeded()  
    }  
}
```

Assumptions: None.

Recommended fix: Consume pointer input on overlays; recycle only after ML Kit tasks finish.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-044

P4 BUG-057 Scanner header detection treats any element containing "100" or "serv" as a header

Confidence: Confirmed **Category:** Data integrity (minor) **Area:** Label scanner

Location: feature/scanner/SpatialTableDetector.kt:397-412

What is wrong: Values such as "1004kJ" or "100mg" push headerBottomY down, so rows above are skipped by the spatial path and fall back to the weaker legacy parser.

How it happens: See "What is wrong".

What the user experiences: Lower-quality parses on some labels.

Impact: Low; tracked for completeness.

Root cause: See "What is wrong".

Evidence:

```
elements.filter { it.text.contains("100") || it.text.contains("serv", ignoreCase = true) }
```

Assumptions: None.

Recommended fix: Only use elements already classified as column headers.

Regression test: Add a unit or lint check for the specific case described.

Related findings: BUG-019

F. User Journey Failures

First install and catalog setup. The built APK requests INTERNET via ML Kit (BUG-008).

Search and log a catalog food (main workflow). Search drops accented and non-Latin letters (BUG-054). Quantity input accepts blank, comma and negative values (BUG-012); rotation resets input (BUG-038). Double taps duplicate entries (BUG-011). The basket editor swallows errors (BUG-039) and lives only in memory (BUG-015). Overnight sessions log to yesterday (BUG-040).

Recipes. Recipes only reach the volatile basket (BUG-015).

Edit or delete past diary entries. Missing fiber, sugar and sodium are stored as 0.0 instead of null in diary snapshots (BUG-037).

Nutrition label scanner. kJ read as kcal (BUG-019), unknown basis treated as per 100 g (BUG-020), per-serving values relabelled per 100 g (BUG-021), thousands separators misread (BUG-022), column confusion (BUG-041, BUG-045, BUG-057), coordinate mixing (BUG-043), warnings lost (BUG-042), gallery OOM crash (BUG-044), overlay touch-through (BUG-056).

Backup, restore and device migration. Restore resets fitness strategy (BUG-016), can wipe a table from an incomplete archive (BUG-017), can crash on large archives (BUG-047), and Merge ignores mineral changes (BUG-051). Android cloud backup copies the personal database regardless (BUG-018); a downgrade wipes data (BUG-046).

Goals, preferences, weight and water. Goal saves can fail silently (BUG-026); a pending strategy cannot be cancelled (BUG-031); preference inputs are dropped or unvalidated (BUG-028, BUG-029); weight editing fails in comma locales (BUG-027); units drift (BUG-050); DataStore errors crash (BUG-053).

Statistics, calendar, rank and widget. Rank is capped near 600 (BUG-023), scored against stale targets (BUG-024) and penalises the current day (BUG-032). Statistics windows are off by one (BUG-033); the calendar recolours history (BUG-049). The widget's tap action is dropped (BUG-030) and it goes stale at midnight (BUG-048).

G. Root-Cause Patterns

Unknown values treated as a convenient default. A missing basis becomes per 100 g (BUG-020), a missing per-100 g value becomes the per-serving value (BUG-021), unparseable quantities become 0 (BUG-012), missing micronutrients become 0.0 (BUG-037), missing backup files become empty tables (BUG-017), missing goal fields become defaults (BUG-016). Each turns "unknown" into a confident wrong number.

Silent failure and success-by-default UI. Results are ignored or errors swallowed (BUG-015, BUG-026, BUG-039, BUG-042), so the user sees success when nothing or the wrong thing was saved.

Locale-blind number handling. String.format without Locale.US paired with toDoubleOrNull parsing produces failures in comma locales (BUG-012, BUG-027, BUG-028, BUG-052).

Tests that do not touch the real implementations. Portability tests use a fake repository, and the merged manifest is never checked. The suite (about 290 tests) passes while the defects above ship.

H. Recommended Fix Order

1. Stop new wrong data from entering the diary (days). BUG-012; BUG-011.

2. Protect history. BUG-037 with a non-destructive migration; BUG-046 (remove the destructive fallback before the next schema change).

3. Make backups trustworthy. BUG-017, BUG-016, then BUG-047 and BUG-051.

4. Restore the privacy promise. BUG-008 (tools:node="remove" plus a CI check) and BUG-018 (backup rules or UI copy).

5. Scanner accuracy. BUG-019, BUG-020, BUG-021, BUG-022, BUG-042, then BUG-041, BUG-043, BUG-045, BUG-044.

6. Recipes and basket completeness. BUG-015, BUG-039.

7. Goals, rank, statistics, widget. BUG-026, BUG-024, BUG-023, BUG-031, BUG-032, BUG-033, BUG-030, then the remaining P3/P4 items.

I. Test Plan

- Edit-mode tests for a FoodDetailViewModel with null secondary nutrients. Loaded and saved values must equal the snapshot unless the user changes quantity, and then scale by newQty / oldQty. (BUG-037)
- Real-repository portability round trip (Room in-memory + fake DataStore): export, archive, validate, Overwrite and Merge import; assert every column and goal state. Negative cases: missing file, count mismatch, oversized archive. (BUG-016, BUG-017, BUG-047, BUG-051)
- Concurrency test: two simultaneous submitBasket calls insert each item once. (BUG-011)
- Input parsing tests under Locale.GERMANY for quantity, weight and preferences. (BUG-012, BUG-027, BUG-028)
- Build check: fail CI if any merged manifest contains android.permission.INTERNET; resource test for backup rules. (BUG-008, BUG-018)
- Scanner golden fixtures: EU kJ/kcal two-column row, ANZ kJ-only, Canadian "Per 1 cup (250 mL)", "Sodium 1,580mg", "Sodium 160mg 7 %", legacy "Calories from Fat", per-serving-only bar; plus an orchestrator test asserting warnings survive adjudication. (BUG-019 to BUG-022, BUG-041, BUG-042, BUG-045)
- Rank replay test over 120 days and a macro-change history test for getGoalForDate. (BUG-023, BUG-024)
- Migration tests with MigrationTestHelper from v1 and v2 (requires exportSchema = true). (BUG-046)

J. Remaining Uncertainty

- Nothing was executed. No build, unit test, instrumented test or app run took place, so every finding is based on reading and tracing code. "Confirmed" means confirmed by code tracing.
- The audit covered the working tree as found: branch master at ddfc057 plus 64 uncommitted modified files and several untracked files (rank, widget, scanner additions). Results may differ from any released build.
- BUG-008 relies on merged manifests from earlier builds (26 Aug release, 8 Sep debug). The dependency is unchanged, so a new build should match, but this was not rebuilt. What ML Kit's transport actually uploads was not observed (no network access).
- Device-specific behaviour (keyboard decimal separator, process-death frequency, OCR element splitting, image sizes) affects BUG-012, BUG-015, BUG-041, BUG-043 and BUG-044; these are marked Highly likely or Potential accordingly.
- The ppocr-sdk module was reviewed only for lifecycle and crash risks; it is not instantiated by the app (BUG-055). The Python data tool and the CSV were read only where they explain the bundled data. androidTest sources were not reviewed in depth.
- No file in the repository contained instructions aimed at an AI auditor that looked malicious; NutritionLabelGeminiAdjudicator.kt contains no network code despite its name.

Priority levels with zero findings: P0.